

PPO 완전 정복 — 서술형 시험 대비 정리

Lecture 12 – Proximal Policy Optimization (3)

TRPO의 한계 → PPO 두 변형 → 구현의 진실 → 활용

이 자료의 사용법 (서술형 대비 전략)

서술형은 "정의"가 아니라 인과(왜 나왔나) → 메커니즘(어떻게 동작하나) → 한계/확장의 연결을 본다.

각 절은 ① 제안 배경 ② 핵심 메커니즘/수식 ③ 의미·한계·확장 순서로 읽으면 답안 구조가 그대로 잡힌다.

마지막 9절의 예상 서술형 7문항 + 답안 뼈대로 출력(output)을 연습할 것. 키워드는 본문의 굵은 글씨.

핵심 한 줄: PPO = "TRPO의 신뢰영역 정신은 계승하되, 2차 최적화를 버리고 1차 최적화(Adam)만으로 근사."

0. 한눈에 보는 큰 그림 (발전 서사)

이 강의 전체는 하나의 이야기로 묶인다. 왜 PPO가 필요했는지(TRPO 한계), 어떻게 그 문제를 풀었으며(두 변형), 실제 성능은 무엇이 만들었는지(Implementation Matters), 지금 어디에 쓰이는지(RLHF)이다.

- 출발점 — TRPO: "정책을 한 스텝에 너무 많이 바꾸지 말자"는 신뢰 영역(trust region) 아이디어를 KL 발산 하드 제약으로 보장. 이론은 우아하나 2차 최적화(자연 그래디언트)에 의존 → 공학적으로 무겁다.
- 전환점 — PPO (OpenAI, Schulman 외, 2017): 같은 정신을 1차 최적화만으로 근사. 슬로건은 "approximately enforce KL constraint without computing natural gradients".
- 두 갈래: (1) Adaptive KL Penalty — 제약을 목적함수의 패널티(soft)로 흡수. (2) Clipped Objective — 확률비(ratio)를 잘라(clip) 과도한 변화를 무력화. 실무 표준은 (2).
- 반전 — "Implementation Matters" (ICLR 2020): PPO의 진짜 성능은 clipping 수식이 아니라 9가지 코드 레벨 최적화에서 나온다는 분석.
- 현재 — 사실상 표준: 안정성·확장성 덕분에 RL의 de facto baseline이자 RLHF로 LLM 정렬의 핵심.

1. 제안 배경 — TRPO의 세 가지 병목

PPO의 존재 이유는 곧 TRPO의 한계다. 세 병목을 "원인 → 타협 → 결과"로 외우면 서술이 쉽다.

병목 1. 2차 최적화의 저주 (The Curse of 2nd-Order Optimization)

- 원인: TRPO는 적응적 스텝 크기를 정하려고 Fisher Information Matrix(FIM)를 2차 테일러 전개로 계산한다.
- 폭발: 정책망 파라미터가 N 개면 FIM은 $N \times N$. 수백만 파라미터의 심층망에서는 행렬 저장과 역행렬 계산이 메모리·연산 폭발을 일으킨다.
- 타협: Conjugate Gradient(CG)로 Hessian-vector product를 근사. 그러나 업데이트마다 반복 루프가 필요해 학습 루프가 받아들이기 힘들 만큼 느려진다.

병목 2. 미니배치 SGD 불가 = 샘플 비효율 (Sample Inefficiency)

- 제약의 성질: 신뢰영역 제약 $D_{KL}(\pi_{old} \parallel \pi_{\theta}) \leq \delta$ 는 현재 롤아웃이 모은 전체 데이터 분포(full batch) 위에서 엄밀히 정의된다.
- 미니배치가 깨지는 이유: 데이터를 쪼개 첫 미니배치로 한 번 업데이트하면 π_{θ} 가 즉시 바뀐다. 그러면 full-batch로 계산한 KL 제약이 즉시 무효/구식이 되어 최적화 경로가 붕괴한다.

- 이론적 약속 vs 현실: 중요도 샘플링(importance sampling, π'/π)으로 과거 데이터 재사용(off-policy)이 이론상 가능하지만, 위 micro-batch 딜레마 때문에 안전성 보장이 파괴된다.

병목 2의 결과 (자주 출제)

TRPO는 사실상 Full-Batch 최적화를 강제당한다: 막대한 버퍼 수집 → 정확히 한 번(1 step) 그래디언트 → 데이터 전부 폐기.

→ 데이터 재사용이 사실상 불가능 → 끔찍한 샘플 효율. PPO가 미니배치 SGD로 K번 재사용하는 것과 정반대.

병목 3. 옵티마이저 비호환성 (Optimizer Incompatibility)

- 모멘텀 vs 하드 컨트롤: 현대 딥러닝은 Adam/RMSProp 같은 1차 옵티마이저(과거 그래디언트의 이동평균 = 모멘텀으로 경로를 부드럽게)에 크게 의존한다.
- TRPO의 경직된 스텝: TRPO는 2차 Hessian/FIM에서 유도된 정확하고 경직된 자연 그래디언트 스텝을 현재 iteration에만 맞춰 강제한다.

$$\Delta\theta = \sqrt{(2\delta / (g^T H^{-1} g)) \cdot H^{-1} g} \text{ (rigid step)}$$

- 결과: 표준 옵티마이저의 내부 상태/모멘텀을 자연스럽게 활용할 수 없다 → 코드가 brittle하고, Transformer 같은 복잡한 구조로 확장하기 매우 어렵다.

2. 이론적 토대 — 왜 "가까운 정책"이면 근사가 통하나

PPO 설계의 정당성은 Lecture 10의 대리 목적함수(surrogate objective)와 상대 정책 성능 한계(relative policy performance bound)에서 나온다.

① 대리 목적함수 $L_{\pi'}(\pi')$

"새 정책의 상태분포 $d^{\pi'}$ 를 옛 분포 d^{π} 로 그냥 근사하면?" 이라는 발상에서 출발한다.

$$J(\pi') - J(\pi) \approx (1/(1-\gamma)) E_{s \sim d^{\pi}, a \sim \pi} [(\pi'(a|s)/\pi(a|s)) \cdot A^{\pi}(s,a)] =: L_{\pi}(\pi')$$

- 이 안에 이미 확률비 π'/π (=PPO의 r_t)가 들어 있다. PPO의 목적함수가 여기서 유래.

② 상대 정책 성능 한계 (식 8)

$$|J(\pi') - (J(\pi) + L_{\pi}(\pi'))| \leq C \cdot \sqrt{E_{s \sim d^{\pi}} [D_{KL}(\pi' || \pi)[s]]}$$

- 해석: 근사 오차의 상한이 두 정책의 KL 발산에 묶인다. 즉 π' 와 π 가 KL로 가까우면 대리 목적함수 근사가 정확하다 — 이것이 "신뢰 영역"의 수학적 근거.
- 상수 C 설정은 강의안에서 "다음 시간에"로 남겨졌고, PPO가 바로 그 실용적 답이다: KL을 직접 풀지 말고 (a) 패널티로 넣거나 (b) ratio를 clip해서 근사적으로 작게 유지.

연결 고리 (이 한 문장이 답안의 허리)

KL이 작으면 근사가 정확(식 8) → TRPO는 이를 하드 제약으로, PPO는 soft 패널티(변형1) 또는 clipping(변형2)으로 "근사적으로" 강제한다.

3. PPO 변형 1 — Adaptive KL Penalty

목적: 제약 최적화 → 비제약(unconstrained) 최적화

TRPO의 하드 제약을 풀지 않고, KL 항을 패널티로 목적함수에 더해 평범한 최대화 문제로 바꾼다.

$$\theta_{k+1} = \arg \max_{\theta} L_{\theta k}(\theta) - \beta_k \cdot D_{KL}(\theta \parallel \theta_k) \quad (\text{식 9})$$

$$D_{KL}(\theta \parallel \theta_k) = E_{s \sim d^{\pi k}} D_{KL}(\pi_{\theta k}(\cdot | s), \pi_{\theta}(\cdot | s)) \quad (\text{식 10})$$

- 페널티 계수 β_k 는 iteration마다 변하며 KL 제약을 근사적으로 강제한다.

알고리즘: PPO with Adaptive KL Penalty

- 입력: 초기 정책 θ_0 , 초기 KL 페널티 β_0 , 목표 KL δ .
- 각 k 에 대해: 부분 trajectory D_k 수집 $\rightarrow$ advantage $\hat{A}_t^{\pi k}$ 추정 $\rightarrow$ 식 9를 Adam으로 K 번 미니배치 SGD 하여 정책 업데이트.
- 업데이트 뒤 측정된 KL로 β 조정:
 - $D_{KL} \geq 1.5\delta$ (너무 많이 변함) $\rightarrow \beta \leftarrow 2\beta$ (페널티 강화)
 - $D_{KL} \leq \delta/1.5$ (너무 조금 변함) $\rightarrow \beta \leftarrow \beta/2$ (페널티 완화)

출제 포인트

초기 β 는 별로 중요하지 않다 — 빠르게 적응하기 때문.

일부 iteration은 KL 제약을 위반할 수 있으나 대부분은 지킨다 ("approximately").

병목 해소 요약: 2차 미분 불필요(병목1 ↓), 미니배치 SGD 가능(병목2 ↓), Adam 사용(병목3 ↓).

4. PPO 변형 2 — Clipped Objective (실무 표준)

확률비를 직접 잘라서, 정책이 한 스텝에 과하게 움직일 유인 자체를 제거한다.

$$r_t(\theta) = \pi_{\theta}(a_t | s_t) / \pi_{\theta k}(a_t | s_t)$$

$$L_{\theta k}^{CLIP}(\theta) = E_{\tau \sim \pi k} [\sum_t \min(r_t(\theta) \cdot \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \cdot \hat{A}_t)]$$

- ϵ 는 하이퍼파라미터(보통 $\epsilon = 0.2$). 업데이트는 $\theta_{k+1} = \arg \max_{\theta} L_{\theta k}^{CLIP}$.

Case 1. 좋은 행동 ($\hat{A}_t > 0$)

- 목표: 정책은 r_t 를 키워 이 성공 행동을 더 자주 선택하려 한다.
- 동작: r_t 가 $1+\epsilon$ (예: 1.2)를 넘으면 clip이 상한을 건다. min은 unclipped($r_t \hat{A}_t$)와 clipped($(1+\epsilon)\hat{A}_t$) 중 작은 값(=clipped)을 고른다 $\rightarrow$ 목적함수에 천장(ceiling)이 생긴다.
- 의미: "보상이 컸더라도 한 스텝에 정책을 과격히 바꿔 망치지 말라." (over-eager 방지)

Case 2. 나쁜 행동 ($\hat{A}_t < 0$)

- 목표: 정책은 r_t 를 줄여 실수를 반복하지 않으려 한다. 그런데 blunder로 r_t 가 $1+\epsilon$ 를 넘어 폭발했다면?
- 동작: clip은 비율을 $1+\epsilon$ 로 cap하지만 이번엔 $\hat{A}_t < 0$. 더 큰 비율 $\times$ 음수 = 더 작은(더 음의) 값. 따라서 min은 clipped를 무시하고 unclipped의 강한 음의 페널티($r_t \hat{A}_t$)를 고른다.
- 의미: "정책이 치명적으로 잘못된 방향이면 clipping이 보호하지 않는다. 페널티 전체를 노출시켜 빠른 교정을 강제한다."

한 줄 핵심 비유 — "보수적 관리자(conservative manager)"

이길 때($\hat{A} > 0$)는 보상에 천장을 씌우지만, 실패할 때($\hat{A} < 0$)는 바닥을 활짝 열어 페널티를 전부 받게 한다.

$\rightarrow$ 비대칭 구조가 PPO clipping의 본질. (서술형에서 이 비대칭을 짚으면 가산점)

5. Check Your Understanding — L^{CLIP} 그래프 해설

질문: 두 그래프(가로축 r) 중 왼쪽/오른쪽이 각각 $A>0$, $A<0$ 중 무엇인가? ($\epsilon \in (0,1)$)

정답: 보기 ① "왼쪽 = $A>0$, 오른쪽 = $A<0$ "

왼쪽 그래프: r 이 커지면 L^{CLIP} 이 증가하다가 $r=1+\epsilon$ 에서 평평(천장) → Case 1 ($A>0$). 비율을 더 키워도 이득이 안 늘게 막은 모습.

오른쪽 그래프: $r < 1-\epsilon$ 구간은 평평하다가, r 이 커지면 계속 아래로 내려감(바닥 없음) → Case 2 ($A<0$). 잘못된 방향일수록 페널티가 무한정 커지는 모습.

빨간 점은 $r=1$ (= 현재 정책 π_{θ_k} , 시작점) 위치.

보기 ③ "ε에 의존" 은 오답: $\epsilon \in (0,1)$ 이면 꺾이는 위치만 달라질 뿐 형태(어느 쪽이 천장/바닥인지)는 A 의 부호로 결정된다.

6. 반전 — "Implementation Matters" (ICLR 2020)

Engstrom, Ilyas 외 (MIT / Two Sigma). 부제: A Case Study on PPO and TRPO. PPO 시험에서 "확장/비판적 이해"를 묻는 단골 소재.

핵심 질문과 발견

- 핵심 질문: PPO의 성공은 정말 L^{CLIP} (clipping) 때문인가, 아니면 잘 보고되지 않은 코드 레벨 최적화 때문인가?
- 놀라운 발견: 수학적 혁신(clipping)의 역할은 의외로 작다. 진짜 동력은 다른 알고리즘 · 휴리스틱에서 빌려온 9가지 코드 레벨 구현 선택이다.
- 증거: clipping을 떼고 코드 트릭만 유지 → 표준 PPO와 대등한 성능. 반대로 트릭 없는 순수 PPO → 성능 급락(catastrophic).

성능을 만든 9가지 코드 레벨 최적화

1. Value function clipping — 원래 $L^V = (V - V_{\text{targ}})^2$. 표준 구현은 PPO식 clip을 적용한 $\max[(V - V_{\text{targ}})^2, (\text{clip}(V, V_{\text{old}} - \epsilon, V_{\text{old}} + \epsilon) - V_{\text{targ}})^2]$ 형태로 가치망을 학습. ϵ 는 ratio clip과 동일값.
2. Reward scaling — 보상을 직접 넣지 않고, 롤링 할인 보상 합을 표준편차로 나눈다(평균은 빠지 않음).
3. Orthogonal initialization + layer scaling — 기본 초기화 대신 층마다 스케일이 다른 직교 초기화 사용.
4. Adam learning rate annealing — task에 따라 (이미 적응형인) Adam의 학습률을 점진 감쇠.
5. Reward clipping — 보상을 preset 범위 보통 $[-5,5]$ 또는 $[-10,10]$ 로 클리핑.
6. Observation normalization — 원시 상태를 평균0/분산1 벡터로 정규화한 뒤 입력.
7. Observation clipping — 관측도 보통 $[-10,10]$ 범위로 클리핑.
8. Hyperbolic tan (tanh) activations — 정책 · 가치망의 층 사이 활성화로 tanh 사용.
9. Global gradient clipping — 정책 · 가치망 그래디언트의 global L2 norm이 0.5를 넘지 않게 클리핑.

서술형에서 쓸 교훈

논문이 내세운 "핵심 수식"과 실제 성능 기여가 다를 수 있다 → ablation, 재현성, 정직한 보고의 중요성.

PPO는 "단일 아이디어의 승리"가 아니라 여러 엔지니어링 선택의 합으로 이해해야 한다.

7. 확장 방향 — PPO의 위상과 활용

- 사실상 표준 baseline: 학계·산업(OpenAI, DeepMind, NVIDIA) 모두에서 1차 비교 기준. 계산 효율·구현 단순성
- 학습 안정성의 균형 때문.

핵심 장점

- Robustness: off-policy(SAC, DDPG) 대비 하이퍼파라미터 안정성이 높고, 다양한 action space에서 더 신뢰할 수 있는 수렴을 보인다.
- Scalability: 1차 그래디언트로 동작하므로 대규모 병렬화(GPU 가속 시뮬레이터, 예: NVIDIA Isaac Sim)와 효율적으로 결합된다. (TRPO 병목 1·3을 피한 직접적 결과)

대규모 AI — LLM 정렬(RLHF)

- PPO는 RLHF(인간 피드백 기반 강화학습)의 핵심 알고리즘으로 LLM을 정렬하는 데 쓰인다.
- 안정성 덕분에 파라미터가 많은 모델을 fine-tuning할 때 정책 붕괴(policy collapse)를 방지한다. — 신뢰영역 정신(작은 KL 변화)이 거대 모델에서 빛을 발하는 지점.

8. 통합 비교표 (TRPO vs PPO-KL vs PPO-Clip)

구분	TRPO	PPO — Adaptive KL	PPO — Clipped
KL 처리	하드 제약 ($D_{KL} \leq \delta$)	soft 패널티 ($-\beta \cdot D_{KL}$)	ratio clip ($[1-\epsilon, 1+\epsilon]$)
최적화 차수	2차 (FIM/자연그래디언트)	1차	1차
미니배치 SGD	불가 (full-batch 1 step)	가능 (K steps)	가능 (K steps)
옵티마이저	자연 그래디언트 (경직된 스텝)	Adam	Adam
데이터 재사용	거의 불가 → 샘플 비효율	재사용 O	재사용 O
구현 난이도	높음 (CG, 역행렬)	중간 (β 조정)	낮음 (가장 단순)
실무 채택	낮음	드물	사실상 표준

표를 한 줄로: "PPO-Clip은 TRPO의 세 병목(2차 최적화·미니배치 불가·옵티마이저 비호환)을 모두 우회하면서 신뢰영역 효과만 남긴 버전."

9. 예상 서술형 7문항 + 답안 뼈대

Q1. TRPO의 세 가지 병목을 설명하고, PPO가 각각을 어떻게 해소하는지 서술하라.

병목1 2차 최적화: FIM($N \times N$) 저장·역행렬 폭발, CG로 근사하나 느림 → PPO는 1차 최적화만 사용.

병목2 미니배치 불가: KL이 full-batch에 정의 → 미니배치 시 즉시 무효화 → full-batch 1 step·데이터 폐기(샘플 비효율). PPO는 패널티/clip으로 K번 미니배치 SGD 재사용.

병목3 옵티마이저 비호환: 경직된 자연그래디언트 스텝이 Adam 모멘텀과 충돌 → PPO는 Adam 그대로 사용, 확장성 ↑.

Q2. 상대 정책 성능 한계(식 8)가 PPO 설계에 주는 의미를 surrogate objective·KL과 함께 설명하라.

$L_{\pi}(\pi')$ 는 $d^{\pi} \approx d^{\pi'}$ 가정 하의 성능 차 근사이며 내부에 확률비 π'/π 를 포함.

식 8: 근사 오차 상한이 $\sqrt{E[D_{KL}]}$ 에 비례 → "KL이 작으면 근사가 정확" = 신뢰영역의 근거.

TRPO는 이를 하드 제약으로, PPO는 soft 패널티(변형1)/clipping(변형2)으로 근사 강제. 상수 C의 실용적 답이 PPO.

Q3. Adaptive KL Penalty의 목적함수와 β 적응 규칙을 쓰고, 비제약 문제로 바꾸는 이점을 설명하라.

목적함수: $\max_{\theta} L_{\theta k}(\theta) - \beta_k \cdot D_{KL}(\theta \parallel \theta_k)$. β 가 KL을 근사적으로 강제.

β 규칙: $D_{KL} \geq 1.5\delta \rightarrow \beta \leftarrow 2\beta$ (강화), $\leq \delta/1.5 \rightarrow \beta \leftarrow \beta/2$ (완화). 초기 β 는 빠르게 적응하므로 덜 중요.

이점: 제약을 패널티로 흡수해 미니배치 SGD+Adam이 가능 $\rightarrow$ 2차 미분 불필요, 데이터 재사용 가능.

Q4. Clipped Objective에서 $A>0$ 과 $A<0$ 일 때 min/clip 동작 차이와 "보수적 관리자" 비유를 서술하라.

$A>0$: r 이 $1+\epsilon$ 넘으면 clip이 상한, min이 작은 값(clipped) 선택 $\rightarrow$ 천장(ceiling). 과도한 정책 변화 방지.

$A<0$: clip이 cap해도 음수 $\times$ 큰비율=더 음수, min이 unclipped(강한 패널티) 선택 $\rightarrow$ 바닥 없음, full penalty.

비유: 이길 때 천장·실패할 때 바닥 개방 = 비대칭. 이 비대칭이 안정적 개선과 빠른 교정을 동시에 달성.

Q5. $L^{\wedge}CLIP$ 그래프(좌/우)를 advantage 부호와 연결해 해설하고, ϵ 의존 여부를 논하라.

왼쪽= $A>0$: 증가 후 $r=1+\epsilon$ 에서 평평(천장). 오른쪽= $A<0$: r 증가 시 계속 하강(바닥 없음). 빨간 점= $r=1$.

$\epsilon \in (0,1)$ 이면 꺾이는 위치만 이동, 형태는 A 부호로 결정 $\rightarrow$ " ϵ 에 의존"은 오답.

Q6. "Implementation Matters"(ICLR 2020)의 핵심 주장·근거와 연구 방법론적 교훈을 논하라.

주장: PPO 성능의 진짜 원인은 clipping이 아니라 9가지 코드 레벨 최적화.

근거: clipping 제거+트릭 유지 $\rightarrow$ 표준 PPO와 대등 / 트릭 없는 순수 PPO $\rightarrow$ 급락.

대표 트릭: value clipping, reward/observation 정규화·클리핑, orthogonal init, tanh, global grad clip, LR annealing.

교훈: 핵심 수식과 성능 기여의 괴리 $\rightarrow$ ablation·재현성·정직한 보고의 중요성.

Q7. PPO가 RLHF/LLM 정렬에서 선호되는 이유를 안정성·확장성 관점에서 서술하라.

안정성: 신뢰영역 정신(작은 KL 변화)으로 고파라미터 모델 fine-tuning 시 policy collapse 방지.

확장성: 1차 그래디언트 $\rightarrow$ 대규모 병렬화·GPU 시뮬레이터(Isaac Sim)와 결합 용이.

off-policy(SAC/DDPG) 대비 하이퍼파라미터 안정성·수렴 신뢰성이 높아 baseline으로 적합.

암기 한 줄 요약 — PPO = "TRPO의 신뢰영역(작은 KL)은 남기고, 2차 최적화·full-batch·자연그래디언트는 버린 1차 근사. 변형1은 KL을 패널티로, 변형2는 ratio를 clip으로 강제. 진짜 성능은 9가지 구현 디테일이 함께 만든다."